

Designing Java Microservices with Spring Boot



Image source: <http://www.freepik.com>

Microservices are polyglot. Developers choose different platforms for building different microservices, as appropriate. Java is still the top choice for developing microservices for enterprises and Spring Boot is the *numero uno* framework for the Java platform. In this sixth part of 'The Design Odyssey' series, we will explore the design aspects of Java microservices by building *AddService* of UMS using the Spring Boot framework.

Once upon a time, software was built and deployed as a single monolith unit. It was difficult to maintain such a monolith in terms of adding and removing features, and fixing the defects. Over a period of time, modular systems replaced monolith systems. However, the modularity was maintained only during the development and not the deployment of software.

For instance, a team of Java developers builds a large e-commerce application as a set of multiple modules like order management, catalogue management, etc. But, the whole code is built together and packaged as a single WAR file. Then the team deploys the WAR file into a Web server like Apache Tomcat. As shown in Figure 1, the monolith WAR needs to be rebuilt and redeployed as a whole, for every change. Because of this, the end user experiences frequent disruptions whenever the team reboots the server for rolling out new features. To avoid this, teams usually delay the feature rollout for weeks and months. Such a delay obviously results in business loss.

The root cause for all this lies in deploying the application as a single monolith WAR. What is the solution?

Microservices

As indicated in the previous part in this series, the solution to the above problem lies in the microservices architecture. If the modules of order management, catalogue management, etc, are not only coded independently but also built and deployed

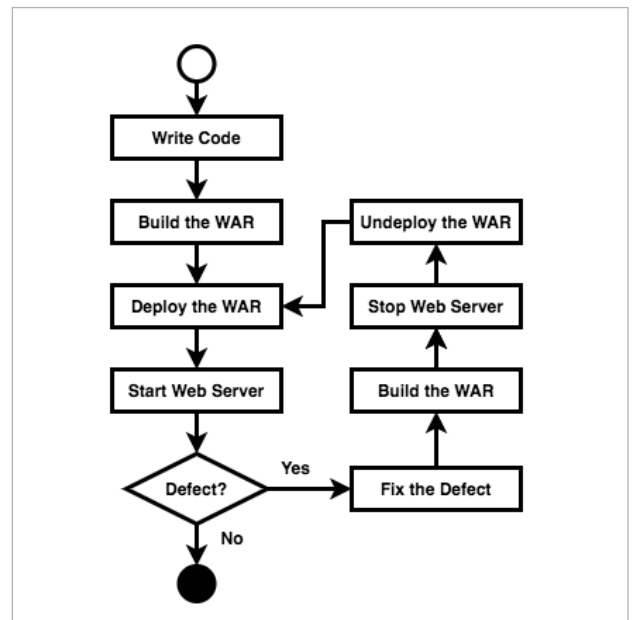


Figure 1: Traditional development and deployment

independently as microservices, the problems cited above will automatically be resolved. This approach allows the team to build and deploy a given microservice any number of times, as frequently as it wishes. The end user will never experience a total shutdown of the e-commerce application.